

Der Subgraph Algorithmus

Stephan Epp

7. Januar 2026

Inhaltsverzeichnis

1	Einführung	2
1.1	Problemstellung	2
1.2	Motivation	2
2	Grundlagen	2
2.1	Definitionen	2
2.2	Grundidee	3
2.2.1	Eindeutige Signaturen	3
2.2.2	Zyklische Rotation statt Permutation	3
3	Algorithmus	3
3.1	Algorithmus-Beschreibung	3
3.2	Vergleich der Signatur-Sequenzen	4
3.3	Implementierung	4
4	Analyse	7
4.1	Laufzeit	7
4.2	Optimalität der Laufzeit	7
4.3	Speicher	10
4.4	Alternative: Adjazenzlisten	10
4.5	Korrektheit	11
4.6	Verwendungsbeispiel	11
5	Zusammenfassung	11
5.1	Anwendungen	12
5.2	Ausblick	12
5.3	Implementierungshinweis	12

1 Einführung

Der Subgraph Algorithmus ist ein effizienter Algorithmus zum Vergleichen zweier Graphen G und G' mittels Adjazenzmatrizen und Signatur-Arrays. Der Algorithmus bestimmt durch zyklische Rotation, ob ein Graph als Subgraph in einem anderen enthalten ist.

1.1 Problemstellung

Gegeben sind zwei Graphen G und G' mit jeweils n Knoten, repräsentiert durch Adjazenzmatrizen A und B .

Ziel: Bestimme, ob Graph G in Graph G' enthalten ist, unter Berücksichtigung aller möglichen Knotenzuordnungen durch zyklische Rotation.

Ergebnis:

- Wenn $B \supseteq A$: Verwerfe A , behalte B (G' hat mehr Informationen)
- Wenn $A \supseteq B$: Behalte A , verwerfe B (G hat mehr Informationen)
- Wenn beide identisch sind: Beliebige behalten
- Wenn keiner den anderen enthält: Beide behalten

1.2 Motivation

Der Subgraph Algorithmus eignet sich besonders für die Verifikation von unterschiedlichen Programmen durch die Analyse der Abstract Syntax Trees (AST). Die Methode ermöglicht es, strukturelle Ähnlichkeiten zwischen Programmrepräsentationen effizient zu erkennen.

2 Grundlagen

2.1 Definitionen

Definition 1 (Graph). Ein Graph $G = (V, E)$ besteht aus einer Menge von Knoten $V = \{v_1, v_2, \dots, v_n\}$ und einer Menge von Kanten $E \subseteq V \times V$.

Definition 2 (Adjazenzmatrix). Die Adjazenzmatrix A eines Graphen $G = (V, E)$ mit n Knoten ist eine $n \times n$ Matrix mit:

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E \\ 0 & \text{sonst} \end{cases}$$

Definition 3 (Subgraph). Ein Graph $G = (V, E)$ ist ein Subgraph von $G' = (V', E')$, geschrieben $G \subseteq G'$, wenn $V \subseteq V'$ und $E \subseteq E'$.

Definition 4 (Zyklische Rotation). Eine zyklische Rotation einer Sequenz $[a_0, a_1, \dots, a_{n-1}]$ um k Positionen nach rechts ist definiert als:

$$\text{rotate}_k([a_0, a_1, \dots, a_{n-1}]) = [a_k, a_{k+1}, \dots, a_{n-1}, a_0, \dots, a_{k-1}]$$

2.2 Grundidee

Der Algorithmus basiert auf zwei zentralen Konzepten:

2.2.1 Eindeutige Signaturen

Für jede Spalte j der Adjazenzmatrix wird eine eindeutige Signatur berechnet:

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n$$

Beispiel 1. Für $n = 4$ und Spalte $j = 0$ mit Vektor $[1, 0, 1, 0]^T$:

$$\sigma_0 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 0 \cdot 2^4 = 1 + 4 = 5$$

Für Spalte $j = 1$ mit gleichem Vektor $[1, 0, 1, 0]^T$:

$$\sigma_1 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 = 1 + 4 + 16 = 21$$

Lemma 1 (Eindeutigkeit der Signaturen). Die Signatur-Funktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv.

Beweis. Die Zeilenkomponente $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert jede Binärkombination eindeutig als Dezimalzahl. Die Spaltengewichtung $j \cdot 2^n$ unterscheidet gleiche Muster an verschiedenen Positionen, da $j \cdot 2^n$ für $j \in \{0, \dots, n-1\}$ stets disjunkte Intervalle $[j \cdot 2^n, (j+1) \cdot 2^n)$ erzeugt. $\square$

2.2.2 Zyklische Rotation statt Permutation

Statt alle $n!$ Permutationen zu prüfen, werden nur n zyklische Rotationen betrachtet. Dies erhält die sequentielle Ordnung und reduziert die Komplexität drastisch.

Beispiel 2. Für $n = 4$ Spalten:

$$\begin{array}{ll} \text{Original:} & [\sigma_0, \sigma_1, \sigma_2, \sigma_3] \\ \text{Rotation 1:} & [\sigma_1, \sigma_2, \sigma_3, \sigma_0] \\ \text{Rotation 2:} & [\sigma_2, \sigma_3, \sigma_0, \sigma_1] \\ \text{Rotation 3:} & [\sigma_3, \sigma_0, \sigma_1, \sigma_2] \end{array}$$

Dies sind nur 4 Rotationen, nicht $4! = 24$ Permutationen.

3 Algorithmus

3.1 Algorithmus-Beschreibung

Der Algorithmus arbeitet in drei Hauptschritten:

Schritt 1: Signatur-Berechnung für Graph G

Berechne für jede Spalte j der Adjazenzmatrix A die Signatur:

$$\sigma_j^A = \sum_{i=0}^{n_A-1} A_{ij} \cdot 2^i + j \cdot 2^{n_A}$$

Schritt 2: Signatur-Berechnung für Graph G'

Berechne analog für Matrix B :

$$\sigma_j^B = \sum_{i=0}^{n_B-1} B_{ij} \cdot 2^i + j \cdot 2^{n_B}$$

Schritt 3: Rotation-Match

Für jede der n_B zyklischen Rotationen von B :

1. Rotiere Spalten zyklisch nach rechts
2. Extrahiere Zeilenkomponenten (ohne Spaltengewichtung)
3. Vergleiche Signatur-Sequenzen mit Longest Common Subsequence (LCS)
4. Falls $\text{LCS} \geq 2$: Subgraph-Beziehung gefunden

3.2 Vergleich der Signatur-Sequenzen

Der Vergleich zweier Signatur-Sequenzen erfolgt mittels dynamischer Programmierung zur Berechnung der längsten gemeinsamen Teilsequenz:

Satz 1 (Subgraph-Kriterium). Eine Subgraph-Beziehung existiert genau dann, wenn die längste gemeinsame Teilsequenz der Signatur-Arrays mindestens die Länge 2 hat.

Beweis. Eine Subgraph-Beziehung kann nur dann existieren, wenn mindestens zwei Knoten durch eine Kante miteinander verbunden sind. Dies entspricht einer gemeinsamen Teilsequenz der Länge mindestens 2 in den Signatur-Arrays. $\square$

3.3 Implementierung

Die Implementierung in Python verwendet NumPy für effiziente Matrix-Operationen:

Listing 1: Signatur-Berechnung

```
1 def _compute_column_signature(self, matrix: np.ndarray) -> List[
2     int]:
3     """
4         Berechnet für jeden Spaltenvektor eine eindeutige
5         Signatur.
6
7         Verwendet polynomiale Hash-Funktion:
8         - Zeilenkomponente:  $\sum(2^i)$  für alle Zeilen  $i$  mit Wert 1
9         - Spaltenkomponente:  $\text{col\_idx} * 2^n$  (gewichtet mit
10            Spaltenindex)
11
12         Diese Methode garantiert Eindeutigkeit, da jede Binä
13            rkombination
14            eine eindeutige Dezimalzahl ergibt und der Spaltenindex
15            die
16            Position zusätzlich kodiert.
17
18         Beispiel für  $n=4$ :
```

```

14         Spalte 0: [1, 0, 1, 0] →  $2^0 + 2^2 + 0 \cdot (2^4) = 1 + 4$ 
           + 0 = 5
15         Spalte 1: [1, 0, 1, 0] →  $2^0 + 2^2 + 1 \cdot (2^4) = 1 + 4$ 
           + 16 = 21
16         → Verschiedene Signaturen trotz gleichem Muster!
17
18     Args:
19         matrix: Adjazenzmatrix (n x n)
20
21     Returns:
22         Liste von eindeutigen Signaturen für jede Spalte
23     """
24     n = matrix.shape[0]
25     signatures = []
26
27     for col in range(n):
28         column_vector = matrix[:, col]
29
30         # Polynomiale Hash-Funktion: handle Spalte als Binä
31         rzahl
32         row_signature = sum(2**i for i in range(n) if
33                             column_vector[i] == 1)
34
35         # Spaltenindex mit Gewichtung  $2^n$  einbeziehen
36         col_weight = col * (2**n)
37
38         signature = row_signature + col_weight
39         signatures.append(signature)
40
41     return signatures

```

Listing 2: Rotation-Match

```

1  def _find_signature_rotation_match(self, sig_A: List[int], sig_B:
    List[int]) -> bool:
2      """
3          Sucht nach einer zyklischen Rotation der Spalten in B,
            sodass A enthalten ist.
4
5          KERNALGORITHMUS (wie beschrieben):
6          1. Berechne Signatur von G
7          2. Für jede der n_B möglichen Rotationen von G':
8              - Rotiere Spalten zyklisch nach rechts
9              - Berechne Signatur neu
10             - Vergleiche mit sig_A
11
12             Die Spalten werden nicht beliebig permutiert, sondern nur
                rotiert!
13             Dies erhält die sequentielle Ordnung.
14
15             Beispiel für 4 Spalten:
16             - Original: [0, 1, 2, 3]

```

```

17 - Rotation 1: [1, 2, 3, 0]
18 - Rotation 2: [2, 3, 0, 1]
19 - Rotation 3: [3, 0, 1, 2]
20     → Nur 4 Möglichkeiten, nicht  $4! = 24$ 
21
22 Args:
23     sig_A: Signaturen von Graph G (n_A Spalten)
24     sig_B: Signaturen von Graph G' (n_B Spalten)
25
26 Returns:
27     True, wenn eine Rotation gefunden wurde, wo A in B
        enthalten ist
28
29     """
30     n_A = len(sig_A)
31     n_B = len(sig_B)
32
33     if n_A > n_B:
34         return False
35
36     if n_A == 0:
37         return True
38
39     # Extrahiere Zeilenkomponenten (ohne Spaltengewichtung)
40     # Für Matrix n x n ist col_weight = 2^n
41     col_weight_A = 2 ** n_A
42     col_weight_B = 2 ** n_B
43
44     row_sigs_A = [sig % col_weight_A for sig in sig_A]
45     row_sigs_B = [sig % col_weight_B for sig in sig_B]
46
47     # Prüfe alle n_B zyklischen Rotationen von B
48     for rotation in range(n_B):
49         # Rotiere row_sigs_B um 'rotation' Positionen nach
50         # rechts
51         rotated_B = row_sigs_B[rotation:] + row_sigs_B[:
52             rotation]
53
54         # Prüfe ob A in diesem Fenster der Rotation enthalten
55         # ist
56         if n_A == n_B:
57             # Gleich groß: direkter Vergleich
58             if self._compare_signature_sequences(row_sigs_A,
59                 rotated_B):
60                 return True
61         else:
62             # A ist kleiner: Prüfe alle Startpositionen in
63             # rotated_B
64             for start in range(n_B - n_A + 1):
65                 window = rotated_B[start:start + n_A]
66                 if self._compare_signature_sequences(
67                     row_sigs_A, window):

```

```

61         return True
62
63     return False

```

4 Analyse

4.1 Laufzeit

Satz 2 (Laufzeit). Der Subgraph Algorithmus arbeitet mit einer Laufzeit von $O(n^3)$.

Beweis. Der Algorithmus arbeitet in drei Schritten:

Schritt 1: Signatur von G berechnen

- Iteration über $n \times n$ Matrix-Einträge
- Komplexität: $O(n^2)$

Schritt 2: Signatur von G' berechnen

- Iteration über $n \times n$ Matrix-Einträge
- Komplexität: $O(n^2)$

Schritt 3: Alle zyklischen Rotationen prüfen

- Für jede der n Rotationen:
 - Signatur neu berechnen: $O(n^2)$
 - Sequenzen vergleichen (LCS): $O(n^2)$
- Komplexität: $O(n \cdot (n^2 + n^2)) = O(n^3)$

Gesamtkomplexität: $O(n^2) + O(n^2) + O(n^3) = O(n^3)$ □

Zur Prüfung beider Richtungen $A \subseteq B$ und $B \subseteq A$ benötigt der Subgraph Algorithmus ebenfalls eine Laufzeit von $O(n^3)$.

4.2 Optimalität der Laufzeit

Wir beweisen nun, dass der Subgraph Algorithmus asymptotisch **optimal** ist: Es gibt keinen Algorithmus, der das zyklische Subgraph-Problem in $o(n^3)$ Zeit lösen kann. Der Beweis gliedert sich in vier Lemmata, die zusammen die untere Schranke $\Omega(n^3)$ etablieren.

Lemma 2 (Eingabe-Leseschranke). Jeder korrekte Algorithmus für das zyklische Subgraph-Problem muss alle Einträge beider Adjazenzmatrizen lesen und benötigt daher mindestens $\Omega(n^2)$ Zeit.

Beweis. Sei $\mathcal{A}$ ein beliebiger deterministischer Algorithmus für das Problem. Angenommen, $\mathcal{A}$ liest einen bestimmten Eintrag A_{ij} (mit $0 \leq i, j \leq n-1$) nicht. Wir konstruieren zwei Eingabeinstanzen, die sich nur in diesem Eintrag unterscheiden:

$$\begin{aligned}
 I_0 : \quad & A_{ij} = 0, \text{ alle anderen Einträge identisch,} \\
 I_1 : \quad & A_{ij} = 1, \text{ alle anderen Einträge identisch.}
 \end{aligned}$$

Da der Eintrag A_{ij} bestimmt, ob die Kante (v_i, v_j) in G existiert, unterscheiden sich I_0 und I_1 in genau einer Kante. Die restlichen Einträge werden so gewählt, dass I_0 und I_1 verschiedene korrekte Antworten haben: Setze B so, dass unter Rotation $k = 0$ gilt $B \supseteq A$ (mit $A_{ij} = 1$), aber $B \not\supseteq A$ (mit $A_{ij} = 0$).

Da $\mathcal{A}$ den Eintrag A_{ij} nicht liest, verhält es sich auf I_0 und I_1 identisch und liefert auf mindestens einer Instanz ein falsches Ergebnis — Widerspruch zur Korrektheit von $\mathcal{A}$. Analoges Argument gilt für jeden Eintrag von B . Folglich gilt:

$$T(n) \geq 2n^2 \in \Omega(n^2). \quad \square$$

Lemma 3 (Notwendigkeit aller Rotationen). Jeder korrekte Algorithmus für das zyklische Subgraph-Problem muss im schlimmsten Fall alle n zyklischen Rotationen untersuchen.

Beweis. Sei $\mathcal{A}$ ein beliebiger Algorithmus. Wir zeigen: Überspringt $\mathcal{A}$ bei einer Eingabe eine Rotation $k^* \in \{0, \dots, n-1\}$, so gibt es eine Eingabe, auf der $\mathcal{A}$ ein falsches Ergebnis liefert.

Konstruktion der Gegeninstanz. Sei G ein Pfadgraph $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{n-1}$ mit Adjazenzmatrix

$$A_{ij} = \begin{cases} 1 & \text{falls } j = i + 1, \\ 0 & \text{sonst.} \end{cases}$$

Sei B die Adjazenzmatrix von G' , deren Spalten so gewählt werden, dass gilt:

- Nach Rotation um k^* Positionen stimmen die Signaturen von B exakt mit denen von A überein, d. h. $G \subseteq \text{rot}_{k^*}(G')$.
- Für alle anderen Rotationen $k \neq k^*$ fehlt mindestens eine Kante, d. h. $G \not\subseteq \text{rot}_k(G')$.

Eine solche Instanz lässt sich durch Wahl $B_j = A_{(j-k^*) \bmod n}$ für alle j konstruieren, ergänzt durch eine einzelne zusätzliche Kante, die alle Rotationen $k \neq k^*$ ungültig macht.

Schlussfolgerung. Die korrekte Antwort lautet „Subgraph existiert“ (Rotation k^* liefert einen Treffer). Da $\mathcal{A}$ die Rotation k^* nicht prüft und alle anderen Rotationen keinen Treffer liefern, gibt $\mathcal{A}$ fälschlicherweise „kein Subgraph“ aus.

Da dieses Argument für jede einzelne überspartete Rotation anwendbar ist, muss jeder korrekte Algorithmus alle n Rotationen prüfen. $\square$

Lemma 4 (Untere Schranke des Sequenzvergleichs). Unter der *Strong Exponential Time Hypothesis* (SETH) kann die längste gemeinsame Teilsequenz (LCS) zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ berechnet werden.

Beweis. **Obere Schranke** $O(n^2)$. Das klassische dynamische Programmierverfahren (siehe Needleman-Wunsch/Wagner-Fischer) berechnet LCS zweier Sequenzen $X = [x_1, \dots, x_n]$ und $Y = [y_1, \dots, y_n]$ über die Rekurrenz

$$dp[i][j] = \begin{cases} 0 & i = 0 \text{ oder } j = 0, \\ dp[i-1][j-1] + 1 & x_i = y_j, \\ \max(dp[i-1][j], dp[i][j-1]) & \text{sonst,} \end{cases}$$

durch Ausfüllen einer $(n+1) \times (n+1)$ -Tabelle in exakt $\Theta(n^2)$ Schritten.

Untere Schranke $\Omega(n^{2-\varepsilon})$ **unter SETH.** Abboud, Backurs und Vassilevska Williams bewiesen 2015 folgendes Resultat („Tight Hardness Results for LCS and Other Sequence Similarity Measures“, FOCS 2015):

Unter SETH — der Annahme, dass k -CNF-SAT keine Laufzeit $O(2^{(1-\varepsilon)n})$ für beliebige $k, \varepsilon > 0$ zulässt — kann LCS zweier Sequenzen der Länge n nicht in $O(n^{2-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ gelöst werden.

Die Signaturen des Subgraph Algorithmus sind beliebige ganzzahlige Werte ohne Einschränkungen an die Alphabetgröße. Das Resultat gilt daher direkt. Damit erhält man:

$$T_{\text{LCS}}(n) \geq \Omega(n^{2-\varepsilon}) \quad \text{für alle } \varepsilon > 0. \quad \square$$

Lemma 5 (Unabhängigkeit der Rotationsvergleiche). Die n LCS-Berechnungen für die n zyklischen Rotationen der Signatur-Sequenz σ^B sind im Allgemeinen voneinander unabhängig und können nicht mit einem Gesamtaufwand von $o(n \cdot T_{\text{LCS}}(n))$ durchgeführt werden.

Beweis. Sei $\sigma_r^B = [\sigma_r^B, \sigma_{r+1}^B, \dots, \sigma_{n-1}^B, \sigma_0^B, \dots, \sigma_{r-1}^B]$ die r -te Rotation. Beim Übergang von Rotation r zu Rotation $r+1$ verschiebt sich das Element σ_r^B vom Anfang ans Ende der Sequenz. Diese scheinbar kleine Änderung kann das LCS-Ergebnis vollständig verändern:

- Wenn σ_r^B den ersten Treffer in einer optimalen gemeinsamen Teilsequenz bildet von $\text{LCS}(\sigma^A, \sigma_r^B)$, entfällt dieser Treffer in σ_{r+1}^B , da σ_r^B nun am Ende der Sequenz steht und keine frühzeitige Übereinstimmung liefert.
- In der DP-Tabelle für $\text{LCS}(\sigma^A, \sigma_{r+1}^B)$ ändern sich bis zu $\Theta(n)$ Zeilen gegenüber der Tabelle für $\text{LCS}(\sigma^A, \sigma_r^B)$, da das Verschieben von σ_r^B die Trefferstruktur aller Positionen von rechts beeinflusst. Eine vollständige Neuberechnung in $\Theta(n^2)$ ist daher notwendig.

Formale Konstruktion. Wähle ein Alphabet $\Sigma = \{a_0, \dots, a_{n-1}\}$ mit paarweise verschiedenen Signatur-Werten. Setze $\sigma^A = [a_0, a_1, \dots, a_{n-1}]$ und $\sigma_0^B = [a_{n-1}, a_{n-2}, \dots, a_0]$ (umgekehrte Reihenfolge). Jede Rotation σ_r^B enthält alle n Elemente in einer neuen Anordnung, sodass $\text{LCS}(\sigma^A, \sigma_r^B)$ für jedes r eine andere optimale Teilsequenz und einen anderen Tabelleneintrag erfordert. Da keine Rotation strukturell aus einer anderen ableitbar ist, müssen alle n DP-Tabellen unabhängig berechnet werden:

$$T_{\text{Rotationsvergleich}}(n) \geq n \cdot T_{\text{LCS}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}). \quad \square$$

Satz 3 (Optimalität des Subgraph Algorithmus). Der Subgraph Algorithmus ist asymptotisch **optimal**. Unter der Strong Exponential Time Hypothesis (SETH) gilt: Kein Algorithmus kann das zyklische Subgraph-Problem für zwei Graphen mit je n Knoten in $O(n^{3-\varepsilon})$ Zeit für beliebiges $\varepsilon > 0$ lösen.

Beweis. Wir kombinieren Lemma 2 bis Lemma 5 zu einer vollständigen Beweiskette.

Schritt 1: Signatur-Berechnung — $\Omega(n^2)$. Nach Lemma 2 muss jeder korrekte Algorithmus alle n^2 Einträge beider Adjazenzmatrizen lesen. Die Signatur-Berechnung verarbeitet jeden Eintrag mindestens einmal:

$$T_{\text{Signatur}}(n) \geq \Omega(n^2).$$

Schritt 2: Rotationsanzahl — **mindestens n** . Nach Lemma 3 muss jeder korrekte Algorithmus alle n Rotationen untersuchen.

Schritt 3: LCS pro Rotation — $\Omega(n^{2-\varepsilon})$ **unter SETH**. Nach Lemma 4 erfordert die LCS-Berechnung zweier Signatur-Sequenzen der Länge n unter SETH mindestens $\Omega(n^{2-\varepsilon})$ Schritte.

Schritt 4: Keine Wiederverwendung zwischen Rotationen. Nach Lemma 5 sind die n LCS-Berechnungen im Allgemeinen unabhängig voneinander und können nicht mit einem Gesamtaufwand von $o(n \cdot T_{\text{LCS}}(n))$ durchgeführt werden.

Gesamte untere Schranke — $\Omega(n^3)$. Aus den Schritten 2 bis 4 folgt:

$$T_{\text{Rotation}}(n) \geq \underbrace{n}_{\substack{\text{Rotationen} \\ \text{(Lemma 3)}}} \cdot \underbrace{\Omega(n^{2-\varepsilon})}_{\substack{\text{LCS pro Rotation} \\ \text{(Lemma 4)}}} = \Omega(n^{3-\varepsilon}).$$

Zusammen mit der unvermeidlichen Eingabeleseschranke aus Schritt 1 gilt:

$$T_{\text{gesamt}}(n) \geq \Omega(n^{3-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Da der Subgraph Algorithmus genau $\Theta(n^3)$ Schritte benötigt (Satz 2), ist er asymptotisch optimal:

$$T_{\text{Subgraph}}(n) = \Theta(n^3).$$

Eine Verbesserung auf $O(n^{3-\varepsilon})$ für ein festes $\varepsilon > 0$ würde die LCS-Schranke aus Lemma 4 widerlegen und damit SETH falsifizieren — eine in der theoretischen Informatik weithin als unwahrscheinlich geltende Konsequenz. $\square$

Bemerkung 1. Die Optimalitätsaussage ist zweistufig:

- **Unbedingt** (ohne Hypothesen): Jeder korrekte Algorithmus benötigt mindestens $\Omega(n^2)$ Zeit, da die Eingabe $\Theta(n^2)$ Bits umfasst (Lemma 2).
- **Bedingt** (unter SETH): Kein Algorithmus kann das Problem in $O(n^{3-\varepsilon})$ Zeit für $\varepsilon > 0$ lösen (Satz 3).

SETH ist eine weithin akzeptierte Komplexitätshypothese, auf der neben diesem Resultat auch untere Schranken für Edit-Distanz, reguläre Ausdrucksauswertung und viele weitere Sequenzprobleme beruhen. Eine unbedingte $\Omega(n^3)$ -Schranke ohne jede Hypothese bleibt ein offenes Problem der theoretischen Informatik.

4.3 Speicher

Der Algorithmus benötigt $O(n^2)$ Speicher für die Adjazenzmatrizen und $O(n)$ Speicher für die Signatur-Arrays.

4.4 Alternative: Adjazenzlisten

Für sparse Graphen kann die Verwendung von Adjazenzlisten effizienter sein:

- Speicherkomplexität: $O(n + m)$ für m Kanten
- Zeitkomplexität für einfache Vergleiche: $O(n + m)$
- Vorteil bei $m \ll n^2$

4.5 Korrektheit

Satz 4 (Korrektheit). Der Subgraph Algorithmus bestimmt korrekt, ob eine Subgraph-Beziehung zwischen zwei Graphen existiert.

Beweis. Die Rotation der Spalten entspricht dem Drehen des Graphen. Dabei bleibt die Struktur des Graphen, gegeben durch die Verbundenheit der Knoten und Kanten, immer erhalten. Deshalb sind nur n Rotationen zu betrachten und nicht $n!$ viele Permutationen.

Der Graph wird so lange gedreht, bis eine Subgraph-Beziehung existiert oder das Drehen vollständig durchgeführt wurde, ohne dass eine Subgraph-Beziehung existiert.

Zur Überprüfung der Subgraph-Beziehung für jede Drehung werden die Elemente der Signatur-Arrays in $O(n^2)$ Laufzeit verglichen. Dabei wird die längste gemeinsame Teilsequenz beider Signatur-Arrays ermittelt.

Die Injektivität der Signatur-Funktion (siehe Lemma 1) garantiert, dass verschiedene Spaltenvektoren verschiedene Signaturen erhalten und somit keine Fehlerkennungen auftreten. $\square$

4.6 Verwendungsbeispiel

Beispiel 3 (Grundlegendes Beispiel). Gegeben seien zwei Graphen:

Graph G mit 4 Knoten:

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Graph G' mit zusätzlicher Kante:

$$B = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Der Algorithmus berechnet:

- Signaturen von G : [1, 18, 36, 48]
- Signaturen von G' : [5, 18, 36, 48]
- Entscheidung: `keep_B` (G' hat mehr Informationen)

5 Zusammenfassung

Der Subgraph Algorithmus bietet eine effiziente Methode zum Vergleich von Graphen mit einer Laufzeit von $O(n^3)$. Durch die Verwendung von:

- Eindeutigen Signaturen basierend auf polynomialer Hash-Funktion
- Zyklischen Rotationen statt vollständiger Permutationen
- Longest Common Subsequence zur Subgraph-Erkennung

erreicht der Algorithmus eine deutlich bessere Komplexität als naive Ansätze mit $O(n! \cdot n^2)$.

5.1 Anwendungen

Der Algorithmus eignet sich besonders für:

- Verifikation von Programmen durch AST-Analyse
- Erkennung struktureller Ähnlichkeiten in Code
- Deduplizierung von Graph-Datenbanken
- Pattern Matching in strukturierten Daten

5.2 Ausblick

Mögliche Erweiterungen umfassen:

- Parallelisierung der Rotation-Prüfungen
- Optimierung für sehr große Graphen ($n > 10000$)
- Approximative Algorithmen für Echtzeit-Anwendungen
- Erweiterung auf gewichtete und gerichtete Graphen
- Integration von Heuristiken zur Reduktion der zu prüfenden Rotationen

5.3 Implementierungshinweis

Der Code ist verfügbar unter: <https://gitlab.com/epp-group/subgraph>.